CECS 326 – Operating Systems

Homework Assignment 3

Due Date: TBD

Objective

The objective of this assignment is to gain hands-on experience implementing several CPU scheduling algorithms in C++. Students will simulate process scheduling, generate Gantt charts, and evaluate scheduling performance using common scheduling metrics.

Learning Outcomes

After completing this assignment, students should be able to:

- Understand CPU scheduling algorithms.
- Simulate process execution.
- Generate Gantt charts.
- Compute waiting time.
- Compute turnaround time.
- Compute response time.
- Compare scheduling algorithms.
- Analyze scheduling performance.

Problem Description

Implement a CPU Scheduling Simulator that executes a set of processes using multiple scheduling algorithms.

Each process contains:

- Process ID
- Arrival Time
- CPU Burst Time
- Priority

The simulator must execute the same workload using the following scheduling algorithms:

- First Come First Serve (FCFS)
- Shortest Job First (Non-preemptive)
- Priority Scheduling (Non-preemptive)
- Round Robin

The program must display a Gantt Chart and compute scheduling statistics for each algorithm.

Programming Requirements

Part 1 – Read Process Information (15 points)

Read process information from a text file.
Each line contains:

- Process ID
- Arrival Time
- CPU Burst Time
- Priority

Example input file:

```
P1  0  8  2
P2  1  4  1
P3  2  9  3
P4  3  5  2
```

Your program should support any number of processes.

Part 2 – FCFS Scheduling (20 points)

Implement the First Come First Serve scheduling algorithm.
Display:

- Gantt Chart
- Waiting Time for each process
- Turnaround Time for each process
- Response Time for each process

Part 3 – Shortest Job First (20 points)

Implement the Non-preemptive Shortest Job First scheduling algorithm.

Display:

- Gantt Chart
- Waiting Time
- Turnaround Time
- Response Time

Part 4 – Priority Scheduling (15 points)

Implement Non-preemptive Priority Scheduling.

Assume:

- Smaller priority value indicates higher priority.

Display:

- Gantt Chart
- Waiting Time
- Turnaround Time
- Response Time

Part 5 – Round Robin Scheduling (20 points)

Implement Round Robin Scheduling.

Requirements:

- Time Quantum = 2

Display:

- Gantt Chart
- Waiting Time
- Turnaround Time
- Response Time

Part 6 – Final Statistics (10 points)

For each scheduling algorithm, display:

```
==========================================
FCFS
==========================================

Gantt Chart

| P1 | P2 | P3 | P4 |

Average Waiting Time:
Average Turnaround Time:
Average Response Time:
CPU Utilization:
```

Repeat the above output for:

- FCFS
- SJF
- Priority Scheduling
- Round Robin

Restrictions

You may NOT use:

- Existing CPU scheduling libraries
- STL `priority_queue`

You MUST use:

- C++
- Linux
- g++
- Arrays or vectors
- A `struct` or `class` to represent a process

Deliverables

Submit a single ZIP file named exactly:

`lastname_HW03.zip`

Inside the ZIP file include:

- `cpu_scheduler.cpp`
- `Makefile`
- `README.pdf`
- Screenshot showing successful compilation
- Screenshot showing successful execution

The README must include:

- Student Name
- Student ID
- Compilation instructions
- Execution instructions

Grading Rubric

Criterion	Points
Correct file input	10
Correct FCFS implementation	20
Correct SJF implementation	20
Correct Priority Scheduling implementation	15
Correct Round Robin implementation	20
Correct scheduling statistics	10
Program compiles successfully using g++	3
Code readability, indentation, comments, organization	2
Total	**100**

Submission Penalties

Violation	Deduction
ZIP file name incorrect	–5
Source file missing	–20
Makefile missing	–10
README missing	–5
Screenshot missing	–5
Program does not compile	Up to –40
Program crashes during execution	Up to –30
Submitted files not inside ZIP archive	–10
Incorrect or extra files causing grading issues	Up to –5

Reflection Questions (Not Graded)

1. What is the purpose of CPU scheduling?
2. What is the difference between waiting time and turnaround time?
3. Which scheduling algorithm may cause starvation?
4. Why does Round Robin improve response time?
5. What is the effect of increasing the time quantum in Round Robin?
6. Why is FCFS considered simple but inefficient?
7. Why does SJF minimize average waiting time?
8. What is response time?
9. Why is Priority Scheduling susceptible to starvation?
10. Explain the trade-off between fairness and throughput.

Extra Credit (+10 Points)

Modify your simulator so that the Round Robin time quantum is specified using a command-line argument.

Example:

```
./scheduler processes.txt 4
```

where

- `processes.txt` = input file
- `4` = Round Robin time quantum

Academic Integrity

All submitted work must be your own. You may discuss concepts with classmates, but the implementation, debugging, and submitted source code must be completed individually.

Students are responsible for ensuring that their programs compile and execute correctly before submission.